

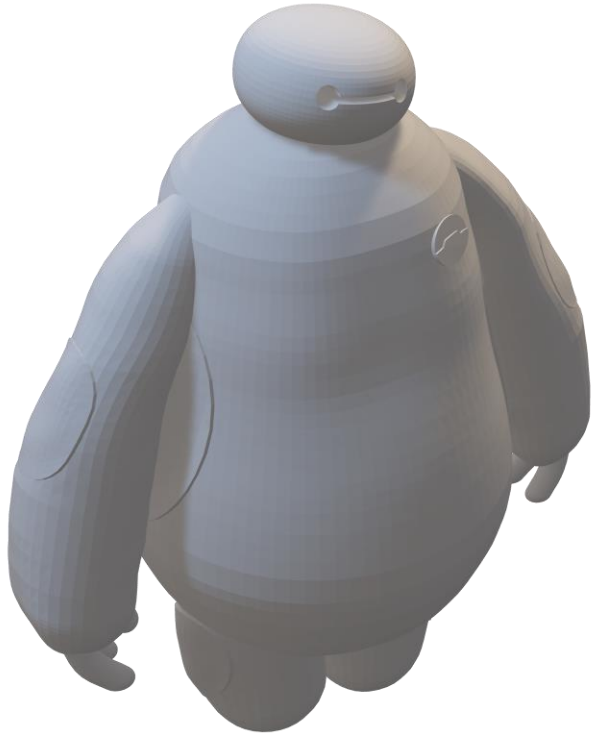
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

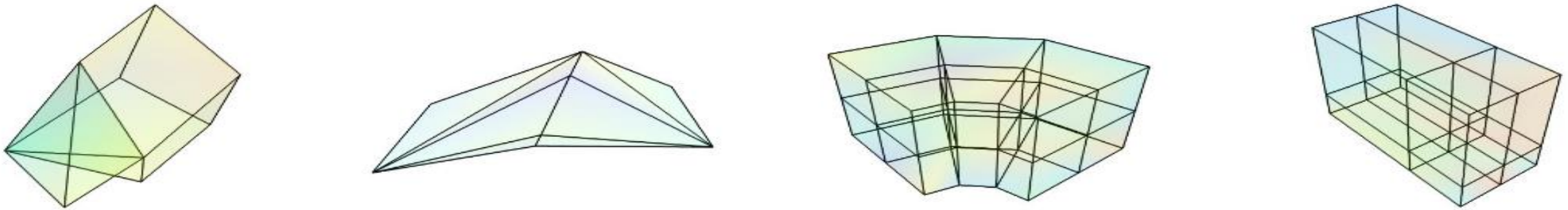
第3章 几何可视化平台搭建

- VTK概述
- Hello VTK
- VTK可视化流程
- VTK适配器构建
- VTK模型变换



1. VTK概述

- **Visullization toolkits（VTK）**是一个开源、跨平台、支持并行处理的免费软件系统，主要用于**计算机图形学、建模、图像处理、体素渲染以及科学可视化**。网址：https://vtk.org/。
- VTK最早由GE研发部的三位研究人员（Will Schroeder、Ken Martin、Bill Lorensen）在闲暇时间合作开发完成。一经发布，VTK很快便建立了用户社区，并迅速在学术研究和商业应用领域得到重用。目前，VTK由Kitware公司维护升级。
- VTK是在三维图形库**OpenGL**的基础上采用面向对象的设计方法发展起来的，其内核使用C++构建，包含大约**250000行代码**，2000多个类，这些类的相互继承关系或类中函数的官方说明文档可在https://vtk.org/doc/nightly/html/index.html在线查看，该文档以C++语言的形式对类或函数进行说明。此外，VTK具有良好的**可移植性**，可以通过Java、Python、Tcl/Tk、C#等其他编程语言自由地使用。
- VTK是一个功能丰富的庞大软件系统，但在Python下，VTK的安装非常简单，无需编译、配置等繁琐易错的工作，我们只需在Windows的cmd窗口输入“**pip install VTK**”命令即可安装，安装完成后可直接使用。目前，VTK最新版本的版本号为8.1.2，安装完成后大小约80MB。

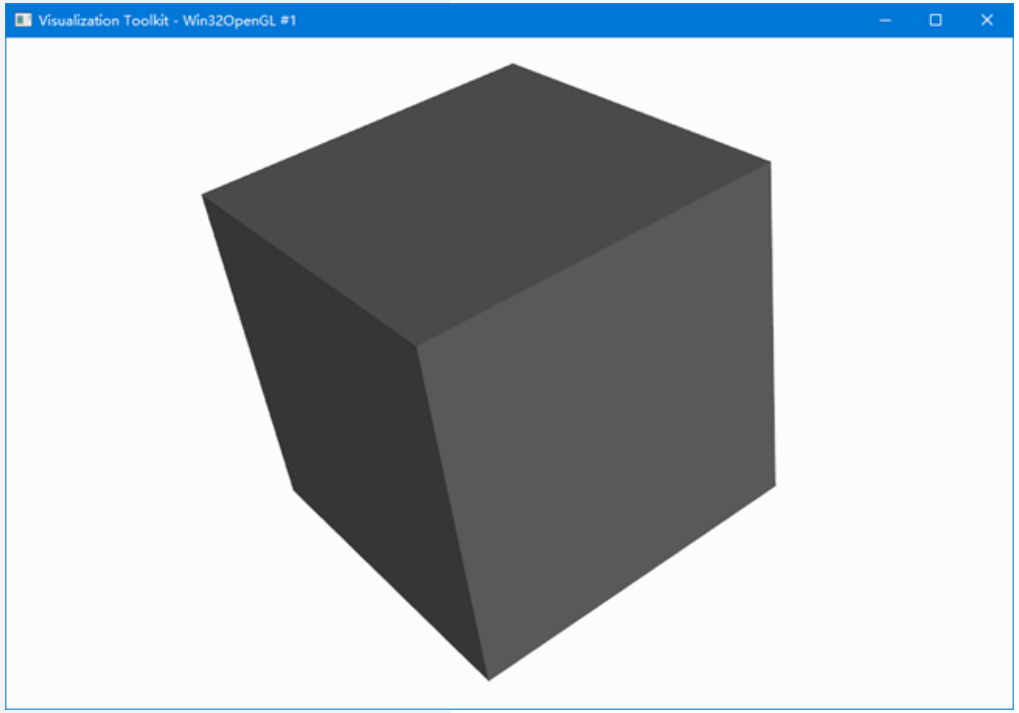


2. Hello VTK

- 在Python中使用VTK编写一个非常简单的类似于“**Hello VTK**”示例程序，用于显示一个正方体，代码如下：

HelloVTK.py | **Hello VTK，利用VTK显示一个长方体** | **3-1**

```
1  import vtk                                                    # 导入vtk库，字母均为小写
2
3  source = vtk.vtkCubeSource()                                  # 创建一个长方体源，默认边长为1,1,1
4
5  mapper = vtk.vtkPolyDataMapper()                              # 创建mapper
6  mapper.SetInputConnection(source.GetOutputPort())             # 设置mapper 数据源
7
8  actor = vtk.vtkActor()                                        # 创建actor
9  actor.SetMapper(mapper)                                       # 设置actor数据源
10 actor.GetProperty().SetColor(0.7, 0.7, 0.7)       # 设置actor显示颜色
11
12 renderer = vtk.vtkRenderer()                                  # 创建渲染器
13 renderer.AddActor(actor)                                      # 将actor添加到渲染器
14 renderer.SetBackground(0.9, 0.9, 0.9)                        # 设置渲染器背景颜色
15
16 window = vtk.vtkRenderWindow()                                # 创建渲染窗口
17 window.AddRenderer(renderer)                                  # 将渲染器添加到渲染窗口
18 window.SetSize(900, 600)                                      # 设置窗口尺寸，单位为像素
19
20 interactor = vtk.vtkRenderWindowInteractor()                  # 创建渲染窗口交互器
21 interactor.SetRenderWindow(window)                            # 提供鼠标、键盘和时钟事件的交互机制
22 interactor.SetInteractorStyle(vtk.vtkInteractorStyleTrackballCamera())    # 设置交互方式
23 interactor.Initialize()                                                   # 初始化交互器
24 interactor.Start()                                                        # 窗口程序开始运行
```



- 可以使用鼠标对三维场景进行操作，如旋转中键滚轮可以放大或缩小视图，按住左键移动鼠标可以旋转视图，按住中键移动鼠标一颗平移视图等。

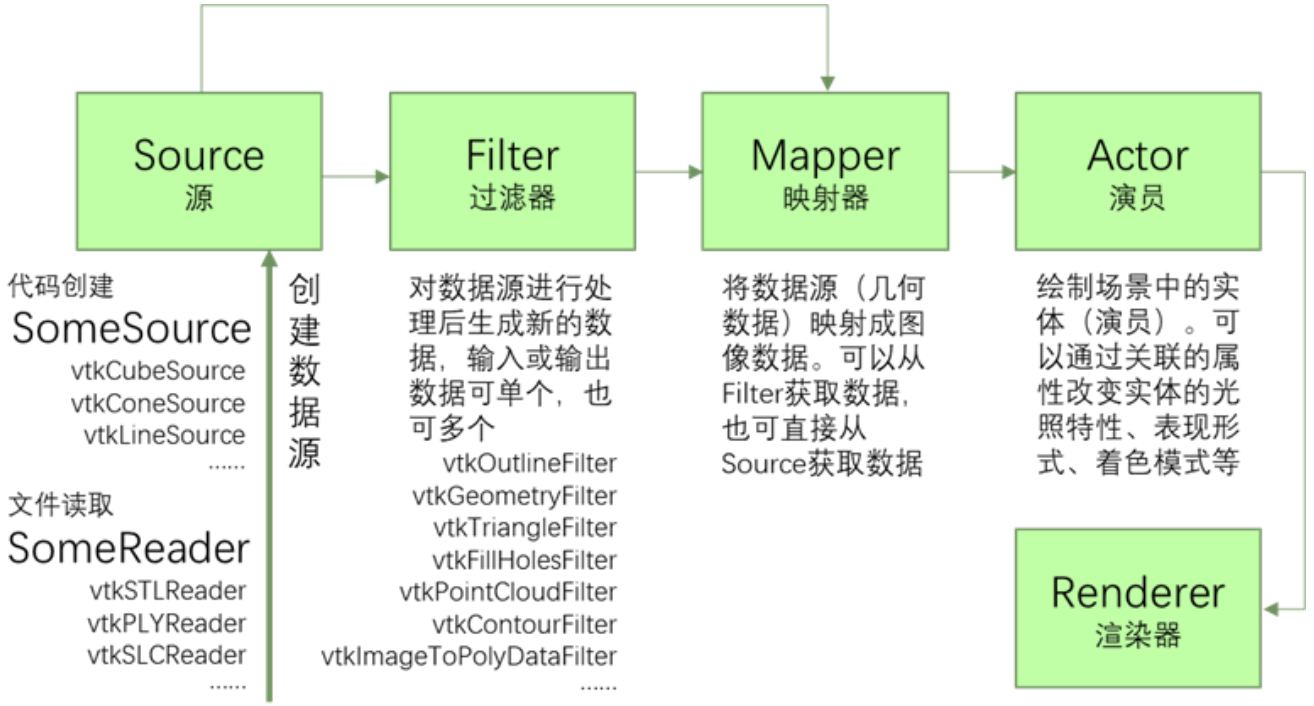
2. Hello VTK

• VTK库使用一般步骤

1. **导入VTK库**。使用import语句引入了VTK库，注意，在实际代码中，vtk为小写字母。
2. **创建source对象**。使用vtk.**vtkCubeSource**()语句创建一个长方体的source对象。vtkCubeSource事实上是一个VTK库中一个类的名字，在VTK库，几乎所有的类名均使用小写的vtk前缀。默认情况下，这条语句将构造一个边长为1，中心点在坐标系原点的立方体；如果需要改变边长或位置，可调用SetXLength、SetYLength、SetYLength以及SetCenter等函数。除了vtkCubeSource外，还可以使用vtkConeSource、vtkCylinderSource、vtkLineSource等类创建圆锥、圆柱、直线等对象。
3. **创建mapper和actor对象**。根据source对象先后创建mapper和actor对象，并且对actor对象设置了颜色属性，这个颜色就是在程序窗口中看到的实体颜色，如有需要，也可以设置actor的其他属性，如透明度、散射等。在VTK中，颜色可使用RGB方式表示和创建，R、G、B每个颜色分量取值为0~1范围内的实数。
4. **创建renderer、window以及interactor对象**。
   - window对象，就是程序的窗口界面，可以使用SetSize函数设置窗口的尺寸。
   - renderer对象，对应窗口中被渲染的视图，可以使用AddActor函数将几何实体的actor对象添加到视图中显示，也可以使用SetBackground函数设置视图背景颜色。
   - interactor对象，其实在每个Windows窗口应用程序的背后，都运行着一个消息循环，用于程序与外部或内部的交互，这里的interactor对象就对应类似的消息处理机制。其SetInteractorStyle函数中设置了“轨迹球相机”的交互方式，这对应用鼠标旋转视图时，相机的位置位于一个轨迹球上。

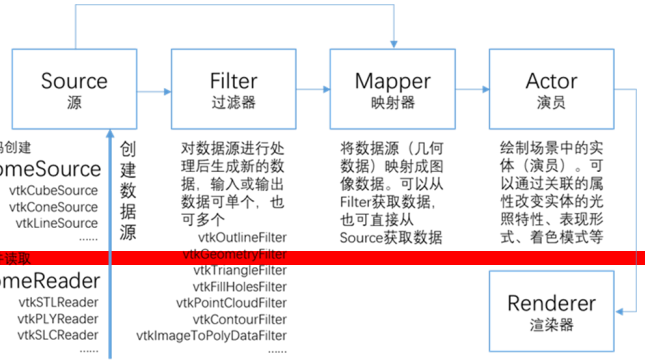
3.VTK可视化流程

- 使用VTK库时，renderer、window以及interactor这3个对象共同构建了三维物体的**显示场景**。一般情况下，场景一旦搭建，就基本保持稳定不变了，经常变化的是场景中的演员，**actor对象就是场景中演员**。actor对象的创建流程如下图所示：



VTK可视化流程

3.VTK可视化流程



（1）创建数据源

- VTK提供了不止一种创建数据源的方法。可以通过代码的方式直接在内存中创建数据源，如vtkCubeSource、vtkConeSource、vtkLineSource等类。对于一些复杂的几何体，还可以通过读取文件的方式获取其数据源，如对三维打印中常用STL模型文件，可以使用vtkSTLReader类来获取模型数据源。

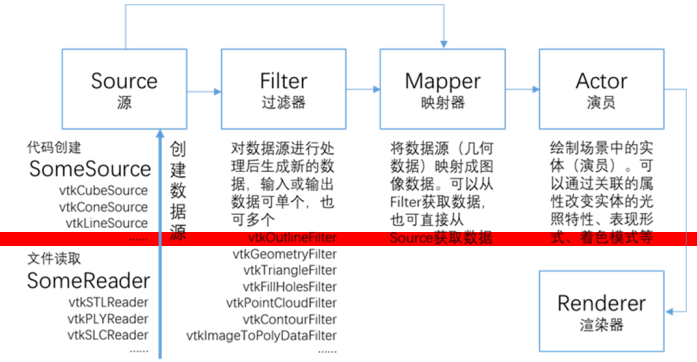
	通过vtkSTLReader创建数据源	3-2
1	stlReader = vtk.vtkSTLReader()	# 创建reader对象
2	stlReader.SetFileName("E:\\3DP.STL")	# 设置模型文件路径

（2）连接过滤器和映射器

- 如果需要对数据源进一步处理，可以将数据源连接到一个过滤器（filter）对象。比如对以上STL模型，如果想从中提取模型的包络矩形边框，就可以将模型数据源连接到一个vtkOutlineFiler的对象。然后连接到映射器。过滤器不是必要的。

	过滤器和数据源连接，映射器和过滤器连接	3-4
1	outlineFilter = vtk.vtkOutlineFilter()	# 创建filter对象
2	outlineFilter.SetInputConnection(stlReader.GetOutputPort())	# filter和source连接
3	mapper = vtk.vtkPolyDataMapper()	# 创建mapper对象
4	mapper.SetInputConnection(outLineFilter.GetOutputPort())	# mapper和filter连接

3.VTK可视化流程



（3）创建演员

- 在获得映射器对象后，可以进一步创建演员（actor）对象。

	演员和映射器连接	3-5
1	actor = vtk. vtkActor ()	# 创建actor对象
2	actor. SetMapper(mapper)	# actor和mapper连接

- vtkActor类中有一个**GetProperty**函数，可以返回vtkProperty类型的对象。通过这个对象，可以修改actor在场景中的外观属性，如实体绘制形式（points、wireframe、surface）、颜色、环境光、散色光、透明度、材料、纹理、边线颜色、顶点颜色、边线线宽等等。

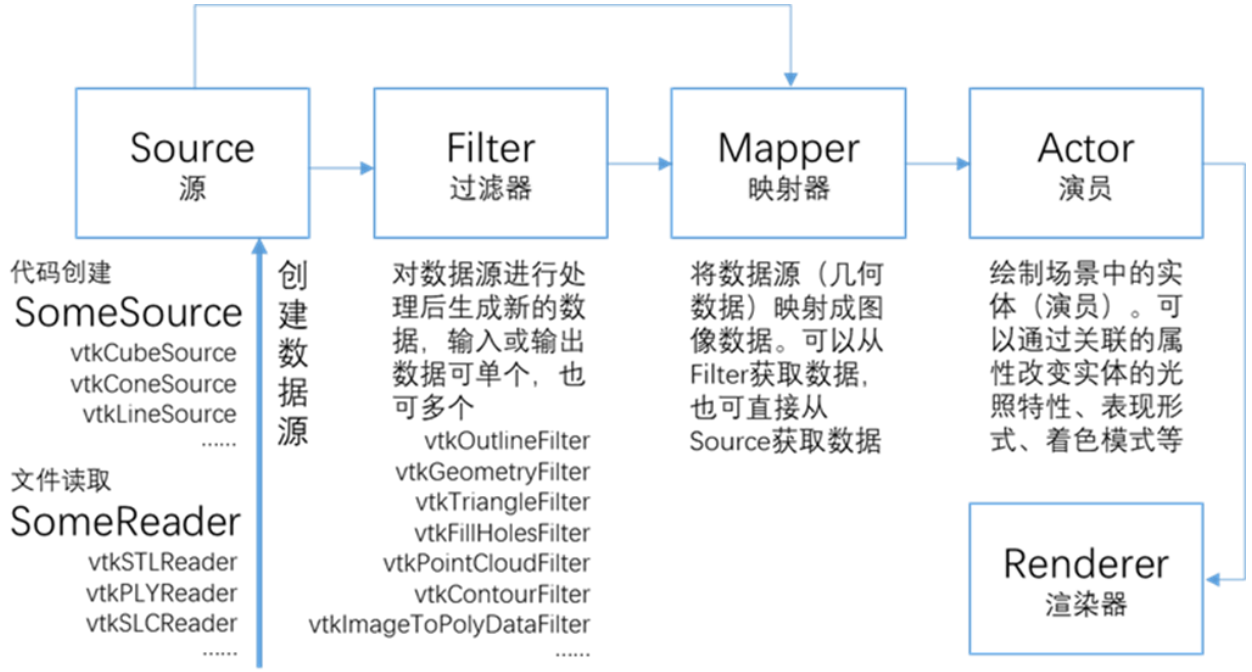
	vtkActor属性（vtkProperty）设置	3-6
1	property = actor.GetProperty	# 获取属性
2	property.SetRepresentationToWireframe()	# 设置线框模型
3	property.SetRepresentationToPoints()	# 设置点模型
4	property.SetRepresentationToSurface()	# 设置面模型
5	property.SetColor(0.5, 0.6, 0.7)	# 设置模型颜色
6	property.SetOpacity(0.5)	# 设置模型透明度
7	property.EdgeVisibilityOn()	# 是否绘制模型边
8	property.SetEdgeColor(0, 0.8, 0)	# 设置模型边界颜色
9	property.SetLineWidth(2)	# 设置线宽
10	……	

3.VTK可视化流程

（4）将演员加入场景

- 获得演员对象后，将演员加入场景。

	将演员加入场景	3-7
1	renderer = vtk.vtkRenderer()	# 创建场景（渲染器）
2	renderer.AddActor(actor)	# 将actor添加到场景



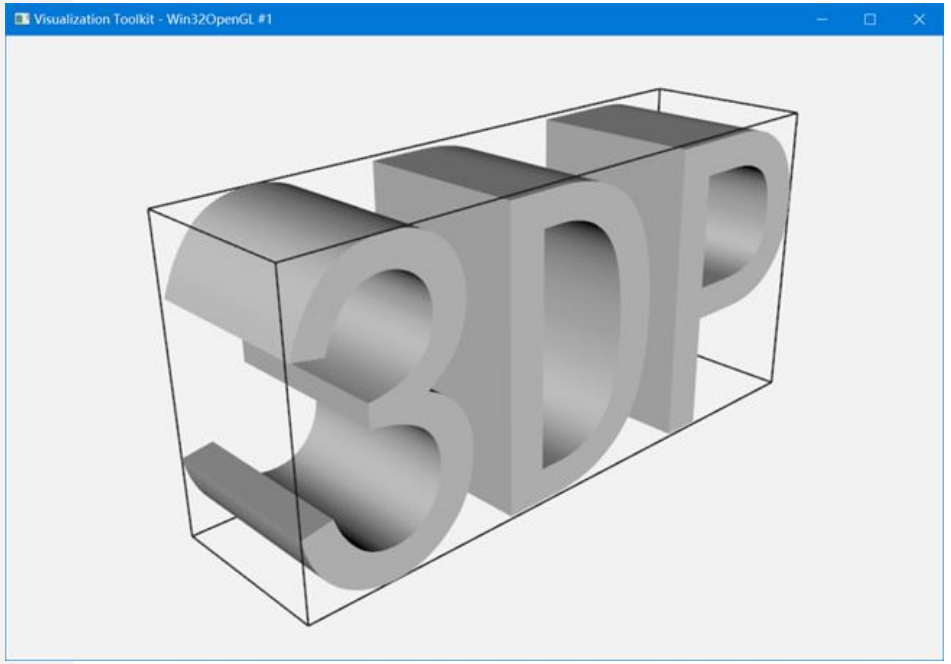
3.VTK可视化流程

- **可视化完整流程**

DisplaySTL.py	利用VTK显示一个STL模型和模型边框（vtkOutlineFilter）	3-8

```
1   import vtk                                          # 导入vtk库
2
3   renderer = vtk.vtkRenderer()                        # 创建三维场景
4   renderer.SetBackground(0.95, 0.95, 0.95)
5   window = vtk.vtkRenderWindow()
6   window.AddRenderer(renderer)
7   window.SetSize(900, 600)
8   interactor = vtk.vtkRenderWindowInteractor()
9   interactor.SetRenderWindow(window)
10  interactor.SetInteractorStyle(vtk.vtkInteractorStyleTrackballCamera())
11  interactor.Initialize()
12
13  stlReader = vtk.vtkSTLReader()                      # 从文件加载STL模型数据源
14  stlReader.SetFileName("E:\\3DP.STL")
15
16  filter = vtk.vtkOutlineFilter()                     # 创建模型边框显示filter-mapper-actor
17  filter.SetInputConnection(stlReader.GetOutputPort())
18  outLineMapper = vtk.vtkPolyDataMapper()
19  outLineMapper.SetInputConnection(filter.GetOutputPort())
20  outlineActor = vtk.vtkActor()
21  outlineActor.SetMapper(outLineMapper)
22  outlineActor.GetProperty().SetColor(0.1, 0.1, 0.1)
23  renderer.AddActor(outlineActor)
24
25  stlMapper = vtk.vtkPolyDataMapper()                 # 创建模型显示mapper-actor
26  stlMapper.SetInputConnection(stlReader.GetOutputPort())
27  stlActor = vtk.vtkActor()
28  stlActor.SetMapper(stlMapper)
29  renderer.AddActor(stlActor)
30
31  interactor.Start()                                  # 运行程序
```

和代码3-1比较，这里调整了部分代码结构，将三维场景的构建放在代码前端。



4.VTK适配器构建

- 如果要显示一个几何实体，**需要写很多和VTK相关的代码**，以构建VTK显示环境。
- 下面用**面向对象编程**的方法和原则，对VTK做的部分功能做进一步封装，得到一个简洁易用的适配器（adaptor）类**VtkAdaptor**。

VtkAdaptor.py　VTK适配器类，对VTK部分函数做进一步封装　3-9

```
1  import vtk                                          # 导入VTK库
2  ……                                                  # 导入必要库
3  class VtkAdaptor:
4      def __init__(self, bgClr = (0.95, 0.95, 0.95)):    # 适配器初始化函数
5          self.renderer = vtk.vtkRenderer()
6          self.renderer.SetBackground(bgClr)
7          self.window = vtk.vtkRenderWindow()
8          self.window.AddRenderer(self.renderer)
9          self.window.SetSize(1000,1000)
10         self.interactor = vtk.vtkRenderWindowInteractor()
11         self.interactor.SetRenderWindow(self.window)
12         self.interactor.SetInteractorStyle(vtk.vtkInteractorStyleTrackballCamera())
13         self.interactor.Initialize()
14     def display(self):                                 # 显示函数
15         self.interactor.Start()
16     def setBackgroundColor(self, r, g, b):             # 设置VTK场景背景色函数
17         return self.renderer.SetBackground(r, g, b)
18     def drawAxes(self, length = 100.0, shaftType = 0, cylinderRadius = 0.01, coneRadius = 0.2):
19         axes = vtk.vtkAxesActor()                      # 绘制坐标轴
20         axes.SetTotalLength(length, length, length)
```

……

4.VTK适配器构建

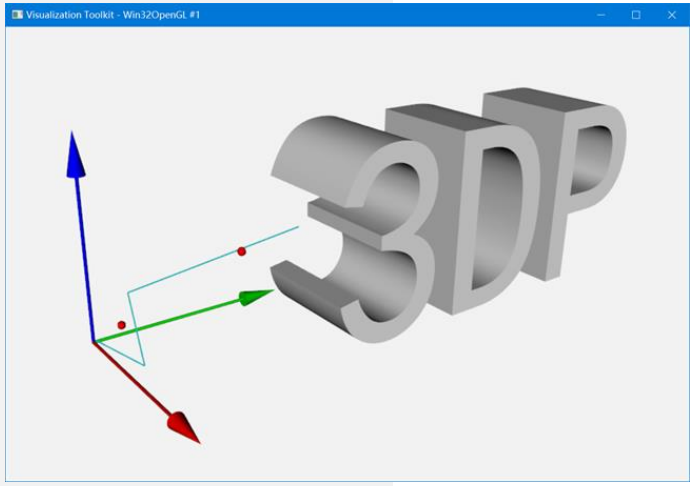
- 在VtkAdaptor类的__init__函数中，通过相应VTK类创建了renderer、window以及interactor属性，因此，在实际应用中，**只需创建VtkAdaptor类的对象，并调用display函数**，就可方便地构建VTK显示场景。如果要改变场景背景颜色，只需调用setBackgroundColor函数即可。
- 在VtkAdaptor类中，封装了若干几何实体绘制函数，包括drawAxes、drawActor、drawPdSrc、以及drawStlModel函数。
  - 函数**drawAxes**的作用是在场景中绘制坐标轴，坐标轴的长度、箭头大小等都可通过该函数参数设置。
  - 函数**drawActor**的作用是绘制vtkActor类型的对象，在调用这个函数时，须确保在此之前已经通过数据源、映射器创建了vtkActor对象。
  - 函数**drawPdSrc**可更加方便地直接对数据源进行绘制，其参数是PolyData类型的source，如通过vtkCubeSource、vtkConeSource以及vtkSTLReader等类型创建的对象。
  - 函数**drawStlModel**对drawPdSrc进一步进行了封装，可直接通过输入STL文件路径绘制STL模型。
- 结合在第2章中创建的几个几何类，封装了drawPoint、drawSegment以及drawPolyline函数。
  - 函数**drawPoint**的输入参数为Point3D对象，可在场景中绘制一个点，点的大小可通过radius参数设置，默认值为2.0。
  - 函数**drawSegment**的作用是绘制线段，输入参数为Segment对象。
  - 函数**drawPolyline**的作用是绘制多段线，输入参数为Polyline。

4.VTK适配器构建

- **VtkAdaptor类测试**

Test_vtkAdaptor.py	VtkAdaptor类测试	3-10

```
1   fromVtkAdaptor import *                    # 导入VtkAdaptor模块
2   if __name__ == '__main__':                 # 测试main函数入口
3       vtkAdaptor = VtkAdaptor()              # 首先创建VtkAdaptor对象，必不可少
4       vtkAdaptor.setBackgroundColor(0.95, 0.95, 0.95)      # 设置场景背景颜色
5       vtkAdaptor.drawAxes()                  # 绘制坐标轴
6       vtkAdaptor.drawPoint(Point3D(10, 10, 10)).GetProperty().SetColor(1, 0, 0)          # 绘制点，设置颜色
7       vtkAdaptor.drawPoint(Point3D(50, 50, 50)).GetProperty().SetColor(1, 0, 0)
8       polyline = Polyline()                  # 创建多段线对象
9       polyline.addPoint(Point3D(1,1,1))      # 在多段线中添加点
10      polyline.addPoint(Point3D(50, 2, 10))
11      polyline.addPoint(Point3D(20, 10, 30))
12      polyline.addPoint(Point3D(50, 80, 55))
13      polylineActor = vtkAdaptor.drawPolyline(polyline)     # 绘制多段线
14      polylineActor.GetProperty().SetColor(0.1, 0.7, 0.7)   # 设置多段线颜色
15      polylineActor.GetProperty().SetLineWidth(2)           # 设置多段线线宽
16      stlActor = vtkAdaptor.drawStlModel("E:\\3DP.STL")  # 根据文件路径绘制STL模型
17      stlActor.SetPosition(0, 150, 150)      # 设置模型位置
18      vtkAdaptor.display()                   # 调用display函数，这一步必不可少
```



5.VTK模型变换

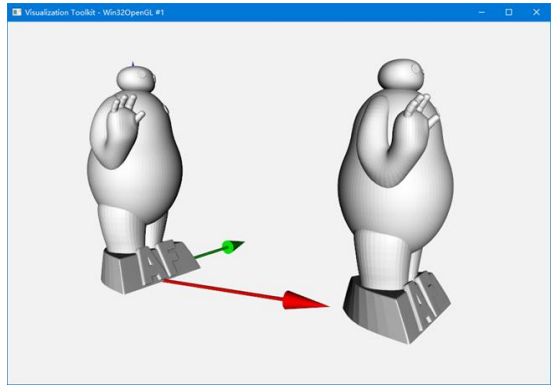
- **vtkActor类继承关系**
  - vtkActor类继承自vtkProp3D类，而vtkProp3D又从vtkProp继承而来。这些类在设计时的功能侧重点不同。
    - **vtkProp类**主要负责实体在场景中渲染层面的工作，包括三维和二维实体渲染，在实际应用中，一般很少直接在代码中调用vtkProp类中的方法。
    - **vtkProp3D类**主要负责实体在场景中几何层面的工作，封装了包括设置/获取实体位置（**Set/GetPosition**）、设置/获取实体缩放比例（**Set/GetScale**）、旋转实体（**RotateX/Y/Z**）以及获取实体包络（**GetBounds**）等功能函数。
    - **vtkActor类**是在编写代码时接触最多、直接使用的类，主要负责实体在场景中显示层面的工作，如**Set/GetProperty**以及**Set/GetTexture**等函数。



5.VTK模型变换

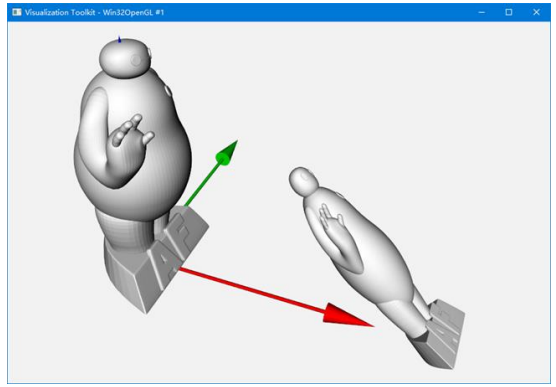
- 模型变换示例
  - 平移变换

	模型平移测试：向X轴正向平移125mm	3-11
1	from VtkAdaptor import *	# 导入VtkAdaptor模块
2	vtkAdaptor = VtkAdaptor(bgClr = (0.95, 0.95, 0.95))	
3	vtkAdaptor.drawAxes()	# 绘制坐标轴
4	actor1 = vtkAdaptor.drawStlModel("E:\\baymax.STL")	# 加载两个模型，actor1作为对照
5	actor2 = vtkAdaptor.drawStlModel("E:\\baymax.STL")	# 对actor2进行平移操作
6	x, y, z = actor2.GetPosition()	# 获取actor2的坐标x、y、z值
7	x += 125.0	# 对x坐标自增125.0mm
8	actor2.SetPosition(x, y, z)	# 对actor2设置更新后坐标
9	vtkAdaptor.display()	



  - 缩放旋转变换

	模型缩放测试：X、Y、Z方向分别缩小2、2、1.5倍	3-12
1	sx, sy, sz = actor2.GetScale()	
2	sx, sy, sz = sx/2.0, sy/2.0, sz/1.1	
3	actor2.SetScale(sx, sy, sz)	
4	actor2.RotateY(-45.0)	



本章小结

- **这一章介绍了一个非常著名的开源几何实体可视化库——VTK。结合Python语言：**
  - 首先展示了一段“Hello VTK”程序，给出了VTK的编程风格和三维场景搭建方法；
  - 在此基础上，分析了VTK从数据源、过滤器、映射器、演员到渲染器的可视化流程；
  - 为了降低VTK编程难度，封装了一个VtkAdaptor类，使用该适配器类可以大大减少VTK在使用过程中的代码语句数量；
  - 最后介绍了vtkActor类的继承关系，并演示了VTK中对模型进行平移、缩放、旋转等几何变换的方法。
- **这一章对于VTK的介绍只是VTK强大功能的冰山一角，对计算机图形学或可视化有强烈兴趣的同学建议阅读VTK官方文档或其C++源码。**





思考题

1. 查阅资料，了解常用的用于矢量图形运算或显示的计算几何库。
2. 简述VTK和OpenGL的关系和区别。
3. 简述VTK中source、mapper和actor的作用和关系。
4. 尝试在VTK中显示矢量形式的文本。
5. 简述VTK的可视化流程。
6. 在VtkAdaptor类中，让draw函数返回actor对象的作用是什么？
7. 尝试实现一个函数，用于读取并显示由AutoCAD绘制保存的DXF文件。